EvoPolicy-SDD: Verificação Orientada por Especificações para a Evolução Segura de Regras de Negócio em Sistemas com Agentes de IA

Lucas Matheus Silva[1]

[1]Instituto Federal da Paraíba (IFPB)
Laboratório de Inovação em Software (LIS)

Abstract. *AI agents that combine Large Language Models (LLMs), document retrieval and deterministic rules increasingly support operational decisions grounded in business policies. When these policies evolve, the retrieved text used as evidence, the requirement specification and the rule actually executed may drift apart without any visible failure. This research proposal presents EvoPolicy-SDD, a specification-driven approach that keeps verifiable links among requirements, executable rules, retrievable documents and logged decisions, and blocks publication when these artifacts diverge. A deterministic prototype (35 automated tests) implements the metamodel, the decision engine, a validator organized into four cumulative levels and an exploratory mutation study. The full level detected 10 of 12 injected drifts with one false alarm among 6 benign changes; the two undetected drifts are semantic and delimit the reach of structural verification. The proposed study extends the change corpus, introduces real LLM-based agents, compares the approach with capability-matched baselines and evaluates LLM-assisted semantic verification.*

Resumo. *Agentes de IA que combinam Modelos de Linguagem de Grande Escala (LLMs), recuperação de documentos e regras determinísticas apoiam cada vez mais decisões operacionais ancoradas em políticas de negócio. Quando essas políticas evoluem, o texto recuperado como evidência, a especificação de requisitos e a regra efetivamente executada podem divergir sem falha visível. Este projeto de pesquisa apresenta o EvoPolicy-SDD, uma abordagem orientada por especificações que mantém vínculos verificáveis entre requisitos, regras executáveis, documentos recuperáveis e decisões registradas, e bloqueia a publicação quando esses artefatos divergem. Um protótipo determinístico (35 testes automatizados) implementa o metamodelo, o motor de decisão, um validador organizado em quatro níveis cumulativos e um estudo exploratório de mutações. O nível completo detectou 10 de 12 derivas injetadas, com um alarme falso em 6 mudanças benignas; as duas derivas não detectadas são semânticas e delimitam o alcance da verificação estrutural. O estudo proposto amplia o corpus de mudanças, introduz agentes com LLMs reais, compara a abordagem com baselines de capacidade pareada e avalia a verificação semântica assistida por LLM.*

1. Introdução

LLMs passaram a ser integrados a sistemas que tomam ou recomendam decisões em processos de negócio, frequentemente na forma de agentes que planejam, consultam ferramentas e produzem respostas justificadas [Yao et al. 2023, Wang et al. 2024]. Para ancorar essas respostas em conhecimento organizacional atualizado, é comum o uso de geração

aumentada por recuperação (*Retrieval-Augmented Generation*, RAG), em que trechos de documentos são recuperados e fornecidos ao modelo como evidência [Lewis et al. 2020, Gao et al. 2023]. Em domínios como reembolso, crédito ou atendimento, contudo, parte das decisões não pode depender apenas da geração de texto: exige regras determinísticas, restrições obrigatórias e uma trilha auditável, inclusive por pressões regulatórias e de governança [National Institute of Standards and Technology 2023, União Europeia 2024, Raji et al. 2020].

O resultado é uma arquitetura híbrida em que convivem ao menos quatro artefatos com ciclos de vida próprios: (i) a especificação de requisitos de negócio; (ii) as regras executáveis que implementam esses requisitos; (iii) os documentos de política usados como evidência pela recuperação; e (iv) o registro das decisões produzidas. Sistemas de software evoluem continuamente [Lehman 1980], e sistemas com componentes de aprendizado de máquina acumulam dívida técnica justamente nas dependências implícitas entre dados, configuração e código [Sculley et al. 2015, Amershi et al. 2019]. Quando uma política muda, nada garante que os quatro artefatos mudem juntos.

O problema investigado neste projeto é a *deriva de coevolução*: a situação em que o documento recuperado para justificar uma decisão, a regra efetivamente aplicada e o requisito que a originou deixam de estar sincronizados após uma alteração de política. Essa deriva é silenciosa. A decisão pode continuar correta enquanto a evidência citada está obsoleta, ou a evidência pode estar atualizada enquanto a regra aplica um limiar antigo. Em ambos os casos a execução não falha, mas a decisão deixa de ser auditável.

Este projeto propõe o *EvoPolicy-SDD*, uma abordagem de Engenharia de Software orientada por especificações versionadas que torna a sincronização entre especificação, regra e documento uma pré-condição verificável de publicação. As contribuições pretendidas são: (i) um metamodelo para coevolução verificável desses artefatos; (ii) um validador com categorias de verificação e níveis de ablação, que permite atribuir a detecção a cada mecanismo; (iii) um conjunto aberto de operadores de mutação e cenários de evolução de políticas; e (iv) evidência empírica sobre ganhos, custos e limites da verificação estrutural e sobre o benefício incremental da verificação semântica. Um protótipo determinístico já implementa (i)–(iii) em pequena escala e fornece os resultados preliminares apresentados na Seção 5.

O projeto é independente e usa um domínio fictício de plataforma de pedidos (reembolsos e cancelamentos), sem dados reais. Uma prova de conceito pública de agente com RAG para atendimento[1] serviu apenas como referência arquitetural; nenhum código ou dado dela é reutilizado. A integração com LLMs reais faz parte do estudo proposto (Seção 6).

O restante do texto está organizado da seguinte forma: a Seção 2 apresenta a contextualização; a Seção 3 discute trabalhos relacionados; a Seção 4 descreve a abordagem proposta; a Seção 5 reporta a avaliação exploratória; a Seção 6 detalha o plano de pesquisa; a Seção 7 discute ameaças à validade; e a Seção 8 apresenta as considerações finais.

2. Contextualização

Agentes baseados em LLM e RAG. RAG combina um recuperador e um gerador para que a resposta seja condicionada a documentos externos [Lewis et al. 2020]; re-

[1] `https://github.com/DouglasKomori/ifood-genai-agent-poc`

visões recentes organizam as variações dessa arquitetura e seus modos de avaliação [Gao et al. 2023]. Agentes estendem esse esquema com raciocínio intercalado a ações e uso de ferramentas [Yao et al. 2023, Wang et al. 2024]. Do ponto de vista de engenharia, sistemas RAG apresentam pontos de falha recorrentes, como conteúdo ausente, recuperação de trechos irrelevantes e extração incorreta [Barnett et al. 2024]; LLMs nem sempre usam bem informações longas no contexto [Liu et al. 2024] e podem produzir justificativas plausíveis sem sustentação nas fontes [Ji et al. 2023].

Especificações como fonte de verdade. Projeto por contrato [Meyer 1992] e engenharia dirigida por modelos [Schmidt 2006] defendem especificações explícitas e verificáveis como fonte do comportamento do software. O desenvolvimento orientado por especificações (*Spec-Driven Development*, SDD) retomou esse princípio para guiar agentes de codificação a partir de especificações versionadas [GitHub 2025]. Neste projeto, o princípio é aplicado a outro alvo: a especificação passa a governar não apenas o código gerado, mas a consistência entre regra executável e texto consumido por um modelo como evidência.

Rastreabilidade e evolução. O problema da rastreabilidade de requisitos foi caracterizado por [Gotel and Finkelstein 1994] e sistematizado em modelos de referência [Ramesh and Jarke 2001]; a rastreabilidade sempre atualizada permanece um desafio em aberto [Cleland-Huang et al. 2014]. Estudos de evolução mostram que artefatos acoplados tendem a divergir quando evoluem em ritmos diferentes, como código e comentários [Tan et al. 2007, Wen et al. 2019], código e documentação [Aghajani et al. 2019] e código de produção e de teste [Zaidman et al. 2011].

Teste de mutação e oráculos. O teste de mutação avalia a capacidade de detecção de um conjunto de verificações injetando defeitos artificiais [DeMillo et al. 1978, Jia and Harman 2011, Papadakis et al. 2019]. A definição do que conta como defeito exige um oráculo, problema clássico em teste de software [Barr et al. 2015]; relações metamórficas [Chen et al. 2018] são uma via complementar para gerar casos sem oráculo explícito.

3. Trabalhos Relacionados

Guardas e regras de negócio. *Frameworks* de guardas programáveis, como o NeMo Guardrails [Rebedea et al. 2023], restringem o comportamento conversacional de LLMs por meio de trilhos declarativos. Regras de negócio têm tradição própria como artefatos declarativos separados do código de aplicação [Ross 2003], e motores de política como o Open Policy Agent [Open Policy Agent sd] tornam políticas executáveis e versionáveis. Essas soluções controlam *o que* o sistema decide, mas não verificam se o texto apresentado como justificativa corresponde à versão da regra aplicada.

Conformidade de processos. A literatura de conformidade de processos de negócio discute como verificar que processos respeitam normas e como lidar com mudanças nessas normas [Hashmi et al. 2018], e critérios de correção para mudanças dinâmicas em *workflows* foram sistematizados por [Rinderle et al. 2004]. Esses trabalhos não consideram agentes que recuperam documentos em linguagem natural como evidência.

Avaliação de RAG e de agentes. Métricas automáticas como as do RAGAs [Es et al. 2024] e o uso de LLMs como avaliadores [Zheng et al. 2023] medem a fidelidade da resposta ao contexto recuperado, mas pressupõem que esse contexto seja a versão correta da política. Revisões sobre teste de sistemas de aprendizado de

máquina [Zhang et al. 2022, Breck et al. 2017] e sobre LLMs em Engenharia de Software [Hou et al. 2024, Fan et al. 2023] não tratam especificamente da coevolução entre regras executáveis e documentos de evidência.

Posicionamento. O EvoPolicy-SDD não propõe um novo motor de regras nem uma nova métrica de fidelidade. Sua contribuição está na interseção desses trabalhos: tornar verificável, antes da publicação, que requisito, regra e documento de evidência continuam consistentes após cada mudança de política, e medir de forma explícita os custos e os limites dessa verificação. A revisão sistemática prevista na Seção 6 [Kitchenham and Charters 2007] confirmará ou refinará esse posicionamento.

4. Abordagem Proposta

Esta seção descreve o metamodelo, o fluxo de verificação e decisão, e as categorias de verificação do EvoPolicy-SDD, tal como implementados no protótipo v0.1.1.

Metamodelo. Cada versão da especificação (`specs/vN/specification.yaml`) declara: o esquema de fatos aceitos (tipos `bool`, `str` e `number`); requisitos com identificador estável e critério de aceitação; regras com prioridade, condição, decisão, justificativa, requisitos atendidos e documento associado; uma decisão padrão; e o *digest* do texto aprovado de cada documento. A base documental (`knowledge/vN/policies.json`) traz versão explícita e, para cada regra vinculada, uma marca de integridade (*fingerprint*) SHA-256 [National Institute of Standards and Technology 2015] calculada sobre os atributos estruturados da regra, incluindo a justificativa.

Fluxo de verificação e decisão. A Figura 1 apresenta o fluxo principal, organizado em seis etapas.

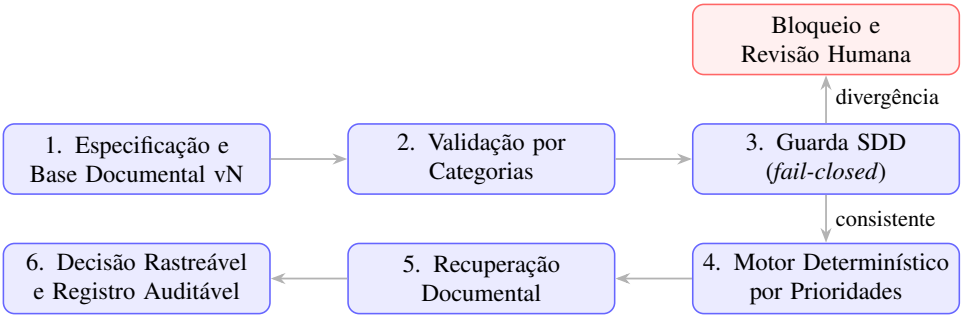


Figura 1. Fluxo de verificação e decisão do EvoPolicy-SDD.

Na *Etapa 1*, a especificação e a base documental de uma versão são carregadas. Na *Etapa 2*, o validador verifica a consistência entre ambas e devolve os problemas encontrados, cada um associado a uma categoria. Na *Etapa 3*, a guarda SDD bloqueia a publicação se houver qualquer divergência e encaminha os casos para revisão humana (política *fail-closed*). Na *Etapa 4*, com os artefatos consistentes, o motor aplica as regras em ordem de prioridade sobre fatos estruturados, usando a decisão padrão da especificação quando nenhuma regra se aplica. Na *Etapa 5*, o documento vinculado à regra é recuperado como evidência. Na *Etapa 6*, a decisão é registrada com a regra, o requisito e o documento citados, além dos *digests* da especificação, da base documental e dos fatos, o que permite *replay* e auditoria. As decisões de projeto do protótipo seguem um registro curto de contexto, decisão e consequências, inspirado em registros de decisões de arquitetura [Nygard 2011].

Categorias de verificação e níveis de ablação. Para que a detecção possa ser atribuída a cada mecanismo, e não ao validador como um todo, cada problema pertence a uma categoria, e os níveis L0–L3 ativam subconjuntos cumulativos dessas categorias (Tabela 1). O nível L1 representa o versionamento convencional que qualquer equipe pode adotar; os ganhos atribuíveis à verificação SDD são, portanto, as diferenças entre L1 e L2/L3.

Tabela 1. Níveis cumulativos do validador.

Nível	Categorias ativas	Interpretação
L0	nenhuma	agente sem guarda
L1	estrutura, versão	versionamento convencional
L2	+ IDs, referências, *fingerprint*	vínculos estruturais regra–documento
L3	+ *digest*, cobertura, esquema	vínculo bidirecional e checagem de tipos

Evolução e análise de impacto. O protótipo oferece ainda operações de diferença entre versões (`diff`), matriz de rastreabilidade requisito–regra–documento (`trace`), recálculo de marcas após revisão humana (`fingerprint`) e o estudo de mutações (`mutate`). Na evolução v1→v2 do domínio de exemplo, a especificação ganha o requisito REQ-05 e a regra REG-PREMIUM-EXC (exceção para cliente *premium* que cancela até 1 hora após o despacho). A análise de impacto identifica exatamente três casos cuja decisão muda (Tabela 2).

Tabela 2. Casos impactados pela evolução v1→v2.

Caso	Decisão v1	Regra v1	Decisão v2	Regra v2
C05	NEGAR	REG-CANCEL-ENVIADO	PERMITIR	REG-PREMIUM-EXC
C06	NEGAR	REG-CANCEL-ENVIADO	PERMITIR	REG-PREMIUM-EXC
C18	NEGAR	REG-CANCEL-ENVIADO	PERMITIR	REG-PREMIUM-EXC

5. Avaliação Exploratória

Os resultados desta seção vêm do protótipo determinístico v0.1.1 (35 testes automatizados) e demonstram propriedades do código escrito. Eles não medem efeitos sobre a qualidade de LLMs e não sustentam inferência estatística: os casos são sintéticos, os rótulos e os mutantes foram escritos pelo próprio autor e há um mutante por família.

5.1. Demonstração com Cinco Condições

Dezoito casos sintéticos de reembolso e cancelamento, rotulados para v1 e v2, foram executados em cinco condições (Figura 2). As condições têm capacidades intencionalmente distintas para demonstrar o mecanismo e não constituem um ensaio causal. A condição B3 (regras v2 com documentos v1) acerta todas as decisões, mas 16 das 18 citam evidência com versão ou *fingerprint* incompatível; as duas restantes não citam documento por usarem a decisão padrão. A mesma configuração sob a guarda SDD (P2) é bloqueada integralmente: troca evidência inválida por indisponibilidade. Com os artefatos sincronizados, a guarda não altera o resultado (B2 e P1 empatam).

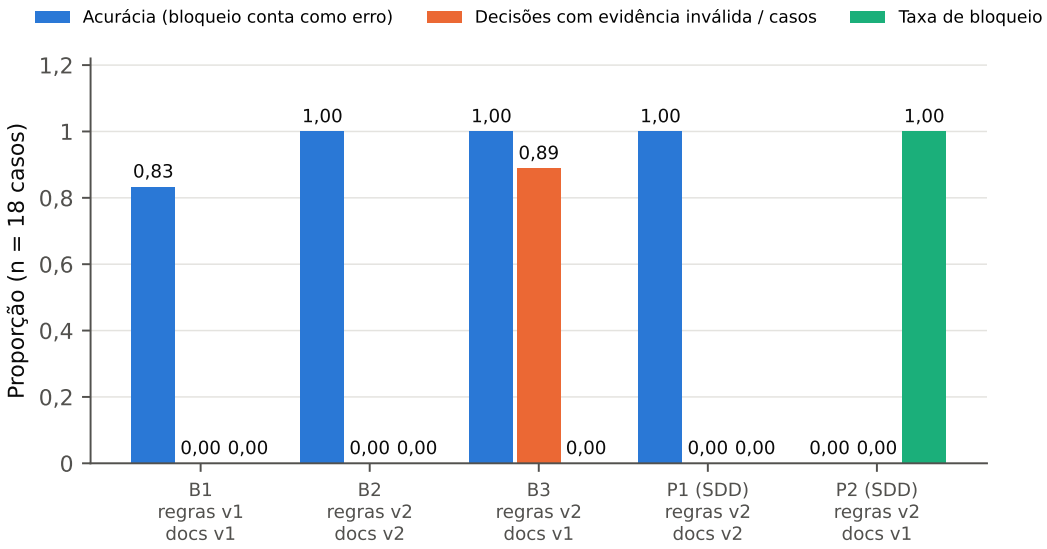


Figura 2. Acurácia, evidência inválida e bloqueio nas cinco condições da demonstração.

5.2. Estudo de Mutações

Partindo do par sincronizado (especificação v2 e documentos v2), doze operadores produzem derivas reais (M01–M12) e seis produzem mudanças benignas (N01–N06), com rótulo de verdade atribuído *a priori* (Tabela 3). Cada mutante é validado nos quatro níveis e os 18 casos são executados com guarda. A Figura 3 mostra o resultado mutante a mutante.

Tabela 3. Operadores de mutação do estudo exploratório.

ID	Deriva injetada	ID	Mudança benigna
M01	limiar alterado só na especificação	N01	reordenação de regras
M02	precedência trocada	N02	reordenação de requisitos
M03	decisão invertida	N03	reordenação de documentos
M04	regra removida	N04	metadados editoriais
M05	requisito removido	N05	correção de pontuação no texto
M06	requisito sem regra	N06	evolução completa e sincronizada
M07	justificativa contraditória		
M08	atualização parcial (um documento antigo)		
M09	texto editado sem revisão		
M10	texto invertido com marcas recalculadas		
M11	limiar alterado com marcas recalculadas		
M12	base documental v1 inteira		

A Figura 4 agrega os resultados. O versionamento convencional (L1) detecta apenas as duas derivas que mudam a versão declarada (M08 e M12). As marcas de integridade (L2) elevam a sensibilidade para 8/12, sem alarmes falsos. O nível completo (L3) chega a 10/12 ao capturar o requisito sem regra (M06) e a edição de texto sem revisão (M09), mas o *digest* de texto também bloqueia uma correção de pontuação (N05), o único alarme falso observado (precisão de 10/11).

A Figura 5 traduz a detecção em efeitos operacionais somados sobre os 18 mutantes. As decisões erradas emitidas caem de 9 (L0 e L1) para 1 (L2 e L3), e as decisões emitidas com evidência divergente caem de 101 para 18. O custo aparece no terceiro painel: em L3, a mudança benigna N05 bloqueia 18 decisões.

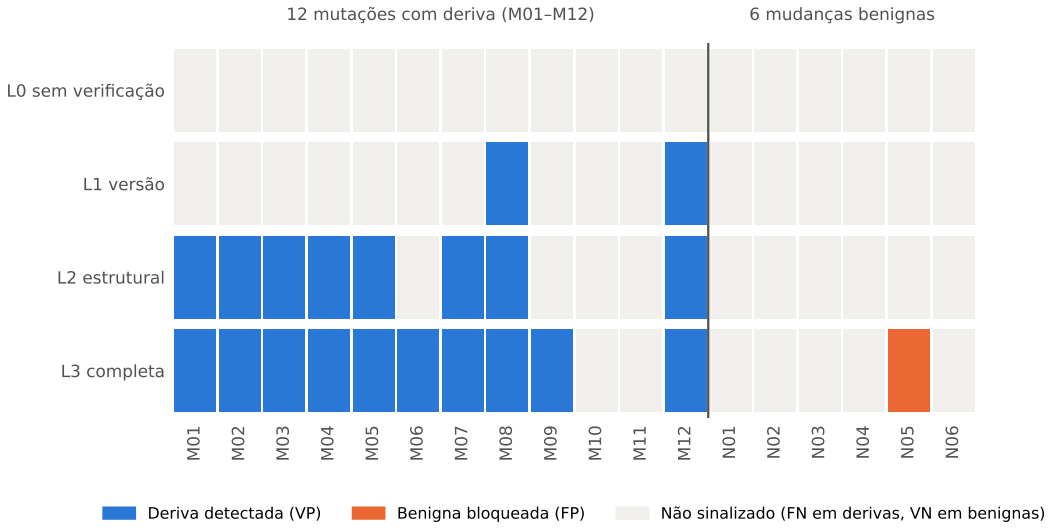


Figura 3. Detecção por mutante e por nível do validador.

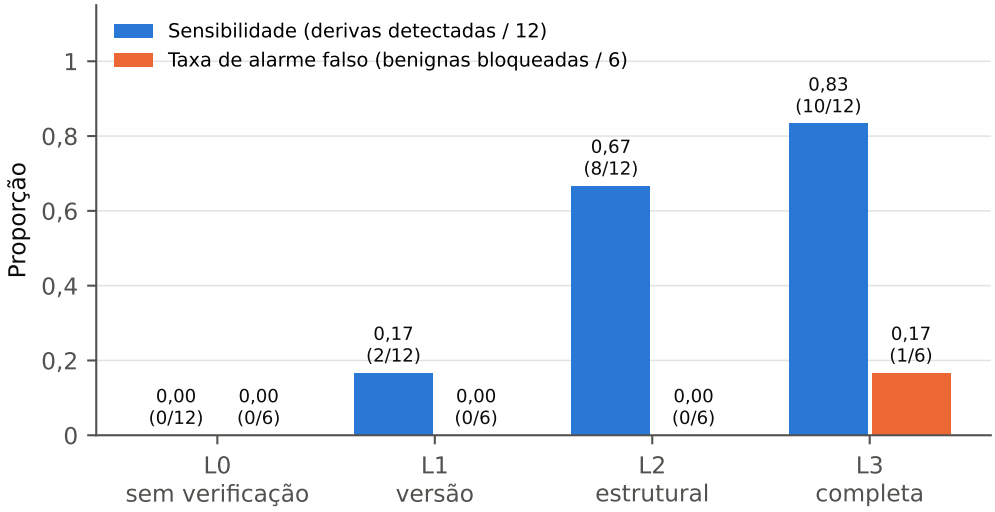


Figura 4. Sensibilidade e taxa de alarme falso por nível do validador.

Limite semântico. M10 e M11 não são detectados em nenhum nível. Em ambos, o significado é alterado (o texto da política é invertido, ou o limiar muda mantendo o texto antigo) e as marcas de integridade são recalculadas sem revisão do conteúdo. A verificação estrutural é contornada por construção: *fingerprints* e *digests* atestam que os artefatos não mudaram desde a última aprovação, e não que texto e regra dizem a mesma coisa. Esse resultado delimita o alcance da abordagem e motiva a principal extensão do estudo: avaliar se um verificador semântico (LLM como juiz, revisão humana, ou ambos) detecta essa classe de deriva e a que custo.

6. Plano de Pesquisa

Questões de pesquisa. A questão principal é: *em que medida um fluxo orientado por especificações versionadas melhora a detecção de inconsistências e a rastreabilidade de decisões de agentes híbridos ao longo da evolução de políticas de negócio?* Ela se

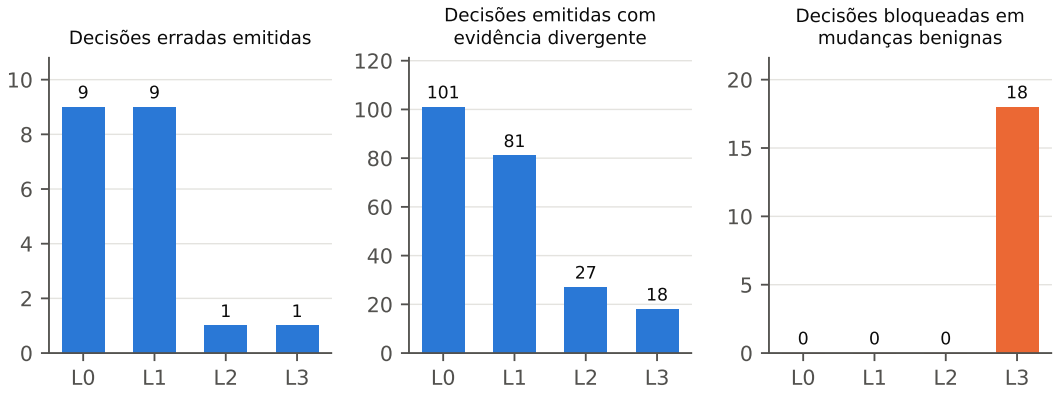


Figura 5. Efeitos operacionais por nível, somados sobre os 18 mutantes.

desdobra em quatro questões:

- **QP1:** a abordagem aumenta a taxa de detecção de divergências entre especificações, regras e documentos, em comparação com *baselines* sem verificação de coevolução?
- **QP2:** a abordagem reduz decisões justificadas com evidência desatualizada durante atualizações controladas de políticas?
- **QP3:** quais custos adicionais a abordagem introduz (bloqueios, manutenção, latência e revisão humana)?
- **QP4:** em quais condições a verificação estrutural deixa passar divergências semânticas, e qual é o benefício incremental de revisão humana ou de verificação assistida por LLM?

As hipóteses são direcionais e orientam a coleta: **H1**, a verificação SDD tem maior sensibilidade do que o versionamento convencional na detecção de deriva; **H2**, ela reduz a proporção de decisões emitidas com evidência versionada incorretamente, com bloqueios contabilizados à parte; e **H3**, ela impõe custos de manutenção e de indisponibilidade mensuráveis e não desprezíveis.

Etapas. O estudo segue as diretrizes de experimentação em Engenharia de Software de [Wohlin et al. 2012] e está organizado em quatro etapas:

1. *Revisão sistemática* [Kitchenham and Charters 2007] sobre *policy-as-code*, conformidade de agentes, evolução de bases RAG, rastreabilidade em sistemas com LLM e SDD.
2. *Corpus de mudanças e operadores de mutação*, com múltiplas políticas e famílias de mudança (exceções, precedência, limiares, documentos conflitantes, inversão semântica com metadados preservados, atualização parcial e requisitos ambíguos), vários mutantes por família e rótulos atribuídos por avaliadores independentes, cegos ao validador.
3. *Integração com LLMs e condições experimentais*, sob o mesmo modelo, corpus, limite de *tokens*, temperatura e tarefas: B1 (agente RAG sem guardas de regra); B2 (agente RAG com regras determinísticas e validação de saída); B3 (B2 com versionamento convencional); P (B3 com verificação SDD, rastreabilidade e bloqueio); e P+S (P com verificação semântica assistida por LLM). O *baseline* B3 permite atribuir os ganhos à verificação SDD, e não apenas ao versionamento; P+S responde a QP4.
4. *Análise*, com sensibilidade e precisão dos alertas, acurácia com bloqueios reportados

à parte, cobertura, validade da evidência, rastreabilidade e custo (latência p50/p95, *tokens*, esforço de atualização e taxa de revisão humana). Intervalos de confiança serão estimados por *bootstrap* agrupado por cenário [Efron and Tibshirani 1993], e comparações pareadas de acerto binário usarão o teste de McNemar [McNemar 1947], com correção para comparações múltiplas. O tamanho amostral será definido por estudo piloto e análise de poder, e o protocolo será pré-registrado.

Cronograma. A Figura 6 apresenta o cronograma previsto para 24 meses. Para doutorado, as etapas 2 a 4 podem ser repetidas em mais domínios e complementadas por estudo de caso em ambiente real [Runeson and Höst 2009].

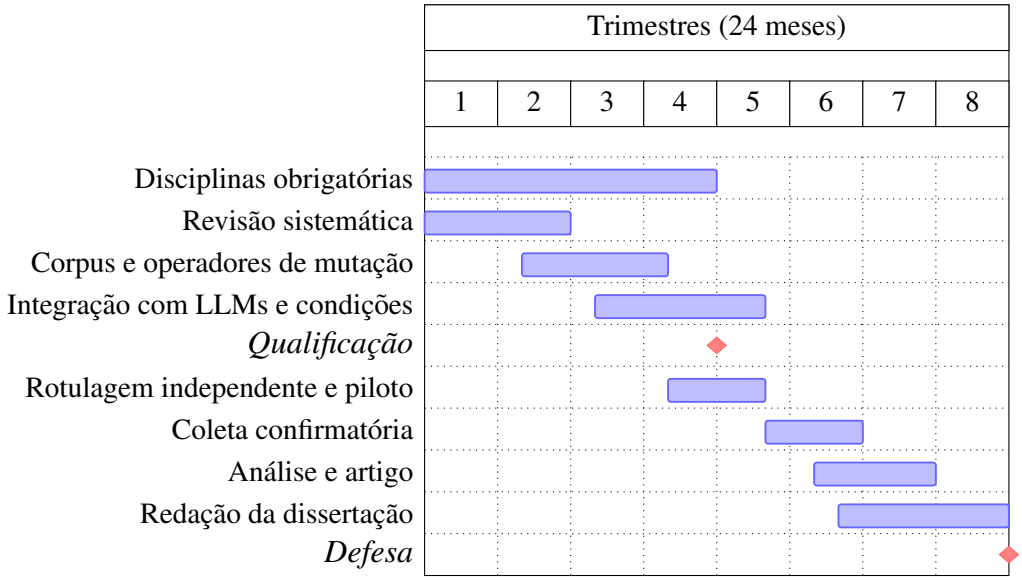


Figura 6. Cronograma previsto.

7. Ameaças à Validade

Em relação à *validade de construto*, *fingerprints* e *digests* verificam estrutura e integridade, não significado, e o oráculo exige revisão independente. Quanto à *validade interna*, um *baseline* simplificado favoreceria a proposta, por isso as condições serão pareadas em funcionalidade e recursos; além disso, mutantes escritos pelo mesmo autor do validador introduzem viés de construção, a ser mitigado com geração sistemática e rotulagem por terceiros. Quanto à *validade externa*, políticas fictícias não representam sistemas regulados reais, e o estudo buscará ao menos um cenário inspirado em documentação pública de um domínio real. Quanto à *validade de conclusão*, casos correlacionados exigem unidade de análise adequada, e o uso de poucos LLMs restringe a generalização. Por fim, do ponto de vista operacional, a política *fail-closed* aumenta bloqueios, cujo custo será medido e reportado.

8. Considerações Finais

Este projeto propõe o EvoPolicy-SDD, uma abordagem orientada por especificações para manter verificáveis os vínculos entre requisitos, regras executáveis, documentos de evidência e decisões de agentes de IA ao longo da evolução de políticas de negócio. O protótipo atual mostra, em pequena escala, que as marcas de integridade explicam a maior parte do ganho sobre o versionamento convencional (de 2 para 8 derivas detectadas em

12), que o *digest* de texto amplia a detecção ao custo de bloqueios por mudanças benignas, e que derivas semânticas com marcas recalculadas escapam a qualquer verificação estrutural.

Do ponto de vista científico, o trabalho pretende contribuir na interseção entre Engenharia de Software, rastreabilidade e sistemas baseados em LLMs, com um estudo controlado sobre ganhos, custos e limites da verificação de coevolução. Do ponto de vista prático, pretende oferecer um mecanismo aplicável a equipes que operam agentes sobre políticas que mudam com frequência, sem substituir a revisão humana. Como próximos passos, estão a revisão sistemática, a ampliação do corpus de mudanças e a integração com LLMs reais, incluindo a avaliação da verificação semântica como resposta ao limite identificado.

Disponibilidade dos Artefatos

O protótipo EvoPolicy-SDD v0.1.1 será disponibilizado como material suplementar e em repositório público. Esse material inclui: (i) código-fonte do pacote `evopolicy` e sua suíte de 35 testes automatizados; (ii) especificações v1 e v2, bases documentais e os 18 casos sintéticos rotulados; (iii) resultados brutos da demonstração e do estudo de mutações (`results/`), incluindo o registro auditável das decisões; e (iv) o *script* que gera as figuras deste texto a partir desses resultados.

Uso de Inteligência Artificial

Em conformidade com o Código de Conduta para Autores da SBC, declara-se que ferramentas de IA foram utilizadas como apoio. O *Claude* (Anthropic) foi utilizado como apoio à implementação do protótipo, à geração das figuras a partir dos resultados e à redação e organização deste texto, sendo o conteúdo científico, a verificação das referências e as conclusões apresentadas de responsabilidade exclusiva do autor.

Referências

Aghajani, E., Nagy, C., Vega-Márquez, O. L., Linares-Vásquez, M., Moreno, L., Bavota, G., and Lanza, M. (2019). Software documentation issues unveiled. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 1199–1210. DOI: `https://doi.org/10.1109/ICSE.2019.00122`.

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., and Zimmermann, T. (2019). Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. DOI: `https://doi.org/10.1109/ICSE-SEIP.2019.00042`.

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., and Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024)*. arXiv:2401.05856.

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., and Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525. DOI: `https://doi.org/10.1109/TSE.2014.2372785`.

Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. In *2017 IEEE*

International Conference on Big Data (Big Data), pages 1123–1132. DOI: https://doi.org/10.1109/BigData.2017.8258038.

Chen, T. Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T. H., and Zhou, Z. Q. (2018). Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1). DOI: https://doi.org/10.1145/3143561.

Cleland-Huang, J., Gotel, O. C. Z., Huffman Hayes, J., Mäder, P., and Zisman, A. (2014). Software traceability: Trends and future directions. In *Future of Software Engineering (FOSE 2014)*, pages 55–69. DOI: https://doi.org/10.1145/2593882.2593891.

DeMillo, R. A., Lipton, R. J., and Sayward, F. G. (1978). Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41. DOI: https://doi.org/10.1109/C-M.1978.218136.

Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall, New York.

Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158. Disponível em: https://aclanthology.org/2024.eacl-demo.16.

Fan, A., Gokkaya, B., Harman, M., Lyubarskiy, M., Sengupta, S., Yoo, S., and Zhang, J. M. (2023). Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. arXiv:2310.03533.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997.

GitHub (2025). Spec kit: toolkit for spec-driven development. https://github.com/github/spec-kit. Acesso em: 11 nov. 2026.

Gotel, O. C. Z. and Finkelstein, A. C. W. (1994). An analysis of the requirements traceability problem. In *Proceedings of the IEEE International Conference on Requirements Engineering (ICRE)*, pages 94–101. DOI: https://doi.org/10.1109/ICRE.1994.292398.

Hashmi, M., Governatori, G., Lam, H.-P., and Wynn, M. T. (2018). Are we done with business process compliance: State of the art and challenges ahead. *Knowledge and Information Systems*, 57(1):79–133. DOI: https://doi.org/10.1007/s10115-017-1142-1.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., and Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8). arXiv:2308.10620.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12). DOI: https://doi.org/10.1145/3571730.

Jia, Y. and Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678. DOI: https://doi.org/10.1109/TSE.2010.62.

Kitchenham, B. and Charters, S. (2007). Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-01, Keele University and Durham University.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076. DOI: https://doi.org/10.1109/PROC.1980.11805.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474. Disponível em: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. DOI: https://doi.org/10.1162/tacl_a_00638.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157. DOI: https://doi.org/10.1007/BF02295996.

Meyer, B. (1992). Applying "design by contract". *Computer*, 25(10):40–51. DOI: https://doi.org/10.1109/2.161279.

National Institute of Standards and Technology (2015). Secure hash standard (SHS). Federal Information Processing Standards Publication FIPS PUB 180-4, NIST, Gaithersburg, MD. DOI: https://doi.org/10.6028/NIST.FIPS.180-4.

National Institute of Standards and Technology (2023). Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, NIST, Gaithersburg, MD. DOI: https://doi.org/10.6028/NIST.AI.100-1.

Nygard, M. (2011). Documenting architecture decisions. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Acesso em: 11 nov. 2026.

Open Policy Agent (s.d.). Open policy agent: Policy-based control for cloud native environments. https://www.openpolicyagent.org/docs. Acesso em: 11 nov. 2026.

Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., and Harman, M. (2019). Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier. DOI: https://doi.org/10.1016/bs.adcom.2018.03.015.

Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., and Barnes, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT* '20)*, pages 33–44. DOI: https://doi.org/10.1145/3351095.3372873.

Ramesh, B. and Jarke, M. (2001). Toward reference models for requirements traceability. *IEEE Transactions on Software Engineering*, 27(1):58–93. DOI: https://doi.org/10.1109/32.895989.

Rebedea, T., Dinu, R., Sreedhar, M. N., Parisien, C., and Cohen, J. (2023). NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445. Disponível em: https://aclanthology.org/2023.emnlp-demo.40.

Rinderle, S., Reichert, M., and Dadam, P. (2004). Correctness criteria for dynamic changes in workflow systems – a survey. *Data & Knowledge Engineering*, 50(1):9–34. DOI: https://doi.org/10.1016/j.datak.2004.01.002.

Ross, R. G. (2003). *Principles of the Business Rule Approach*. Addison-Wesley, Boston.

Runeson, P. and Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164. DOI: https://doi.org/10.1007/s10664-008-9102-8.

Schmidt, D. C. (2006). Model-driven engineering. *Computer*, 39(2):25–31. DOI: https://doi.org/10.1109/MC.2006.58.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, pages 2503–2511. Disponível em: https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.

Tan, L., Yuan, D., Krishna, G., and Zhou, Y. (2007). /* iComment: Bugs or bad comments? */. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 145–158. DOI: https://doi.org/10.1145/1294261.1294276.

União Europeia (2024). Regulamento (UE) 2024/1689 do parlamento europeu e do conselho, de 13 de junho de 2024, que cria regras harmonizadas em matéria de inteligência artificial (regulamento da inteligência artificial). Jornal Oficial da União Europeia, Série L, 12 jul. 2024. Disponível em: https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6). DOI: https://doi.org/10.1007/s11704-024-40231-1.

Wen, F., Nagy, C., Bavota, G., and Lanza, M. (2019). A large-scale empirical study on code-comment inconsistencies. In *Proceedings of the 27th IEEE/ACM International Conference on Program Comprehension (ICPC)*, pages 53–64. Disponível em: https://www.inf.usi.ch/faculty/lanza/PUBS/P/Wen2019a.pdf.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg. DOI: https://doi.org/10.1007/978-3-642-29044-2.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*. arXiv:2210.03629.

Zaidman, A., Van Rompaey, B., van Deursen, A., and Demeyer, S. (2011). Studying the co-evolution of production and test code in open source and industrial developer test processes through repository mining. *Empirical Software Engineering*, 16(3):325–364. DOI: https://doi.org/10.1007/s10664-010-9143-7.

Zhang, J. M., Harman, M., Ma, L., and Liu, Y. (2022). Machine learning testing: Survey, landscapes and horizons. *IEEE Transactions on Software Engineering*, 48(1):1–36. DOI: https://doi.org/10.1109/TSE.2019.2962027.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*. arXiv:2306.05685.